Low Level Design

On

Telecomm Quiz/Tutor Bot

Course Name: Gen AI

Institution Name*:* Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Mohd Dilshad Khan	EN22CS301602
2.	Nishit Sharma	EN22CS301658
3.	Paridhi Gupta	EN22CS301683
4.	Nirmal Kant Shrivastava	EN22CS301656
5.	Mohd Suhail Khan	EN22CS301604

Group Name: Group 08D6

Project Number: GAI-19

Industry Mentor Name:

University Mentor Name: Prof. Ajaj Khan

Academic Year: 2025-2026

Table of Contents

1. Introduction

1.1. Scope of the document

1.2. Intended Audience

1.3. System overview

2. System Design

2.1. Application Design

2.2. Process Flow

2.3. Components Design

2.4. Key Design Considerations

2.5. API Catalogue

3. References

1. Introduction

The Telecom Quiz/Tutor Bot is an AI-driven educational system designed to enhance users' understanding of telecom concepts through interactive learning. The system allows users to attempt quizzes related to telecom topics such as prepaid and postpaid plans, data usage, network technologies, and telecom services.

Unlike traditional quiz systems, this application uses a Large Language Model (LLM) to intelligently evaluate user responses. Instead of simply marking answers as correct or incorrect, the system provides detailed feedback, explains mistakes, and reinforces the correct concepts. This creates a personalized and effective learning experience for users.

In addition to quizzes, the system also includes a chatbot feature that can answer telecom-related queries in real time. The chatbot leverages AI and a vector database to provide context-aware and accurate responses.

The project is developed using modern technologies such as Python (FastAPI) for backend development, LLM APIs for intelligent evaluation, and Vector Databases for knowledge retrieval. The system is designed to be scalable, efficient, and user-friendly.

Overall, the Telecom Quiz/Tutor Bot aims to bridge knowledge gaps in the telecom domain by combining AI-powered evaluation, interactive quizzes, and conversational learning into a single platform.

1.1 Scope of the Document

This document provides a detailed low-level design of the **Telecom Quiz/Tutor Bot** system. It focuses on the internal structure, components, and interactions required to implement the application effectively.

The document covers:

- The overall system architecture and module breakdown
- Application design including frontend, backend, and AI processing layers
- Process flow of quiz evaluation and chatbot interaction

- Integration of Large Language Models (LLMs) for intelligent answer evaluation
- Use of vector databases for knowledge retrieval in chatbot functionality
- API design and communication between system components

The scope of this document is limited to the design and implementation aspects of the system. It does not include detailed user interface design, deployment infrastructure, or advanced third-party integrations.

1.2 Intended Audience

This document is intended for all stakeholders involved in the development, implementation, and evaluation of the Telecom Quiz/Tutor Bot system.

It is useful for:

- Developers – To understand the system architecture, modules, APIs, and implementation details required to build the application.
- AI Engineers – To understand how Large Language Models (LLMs) are integrated for answer evaluation and chatbot functionality.
- Project Evaluators / Faculty – To review the design, structure, and technical approach of the system.
- Students / End Users – To gain a basic understanding of how the system works and how it provides personalized learning through quizzes and AI-based feedback

1.3 System Overview

The **Telecom Quiz/Tutor Bot** is an AI-powered system designed to provide an interactive and intelligent learning experience in the telecom domain. The system allows users to attempt quizzes on various telecom topics such as prepaid and postpaid plans, data usage, network technologies, and telecom services.

When a user submits an answer, the system processes it through a backend API and evaluates it using a **Large Language Model (LLM)**. The model analyzes the response, determines its correctness, and generates detailed feedback, including explanations of mistakes and clarification of concepts. This approach goes

beyond traditional rule-based systems by providing contextual and meaningful evaluation.

In addition to the quiz functionality, the system includes a chatbot that can answer telecom-related queries in real time. The chatbot uses a combination of LLM and a vector database to retrieve relevant information and generate accurate, context-aware responses.

The system follows a modular architecture consisting of a frontend interface, backend API layer, AI processing layer, and data storage. This design ensures scalability, maintainability, and efficient handling of user interactions.

Overall, the system aims to improve users' telecom knowledge through **AI-driven evaluation, personalized feedback, and conversational assistance**, making learning more engaging and effective.

2. System Design

The **Telecom Quiz/Tutor Bot** system is designed using a modular and layered architecture. It consists of a frontend interface, backend API, AI processing layer, and data storage.
The frontend allows users to attempt quizzes and interact with the chatbot. The backend, built using FastAPI, handles requests and communicates with the AI components.
The processing layer uses a **Large Language Model (LLM)** to evaluate user answers and generate feedback. The chatbot uses a vector database along with the LLM to provide accurate responses.
The system ensures efficient interaction between components, enabling real-time evaluation and personalized learning.

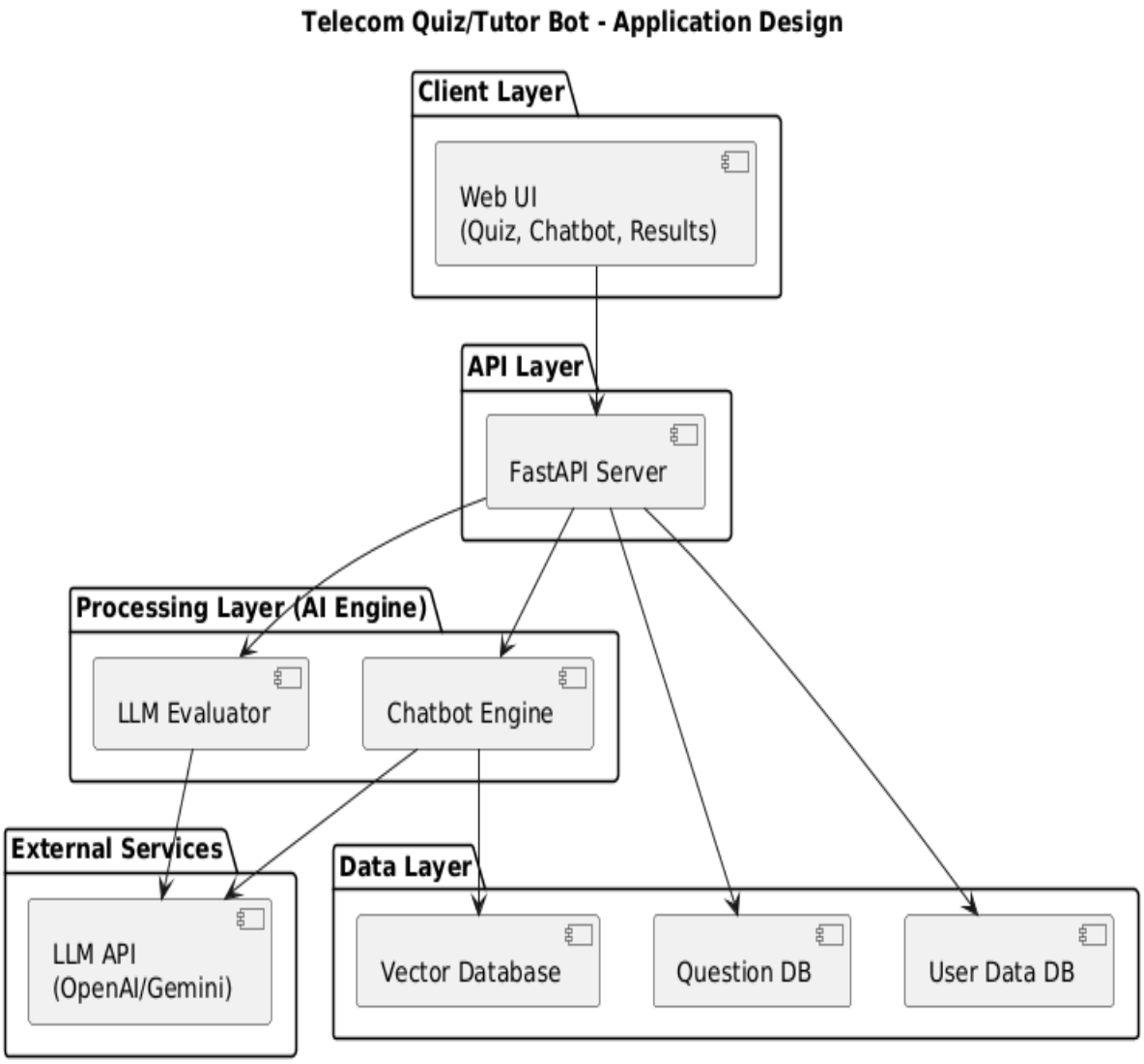
2.1 Application Design

The application is designed using a layered architecture consisting of the following layers:

1. **Client Layer**
    - Users interact with the system through a web interface.
    - It includes quiz, chatbot, and result display features.
2. **API Layer**
    - Built using FastAPI.

- Handles incoming requests and routes them to the processing layer.

3. **Processing Layer (AI Engine)**
   - Evaluates user answers using a Large Language Model (LLM).
   - Generates scores and feedback.
   - Handles chatbot queries.
4. **Data Layer**
   - Stores questions, user responses, and results.
   - Uses a vector database for knowledge retrieval.
5. **Deployment Layer**
   - Runs the backend application and integrates LLM APIs.



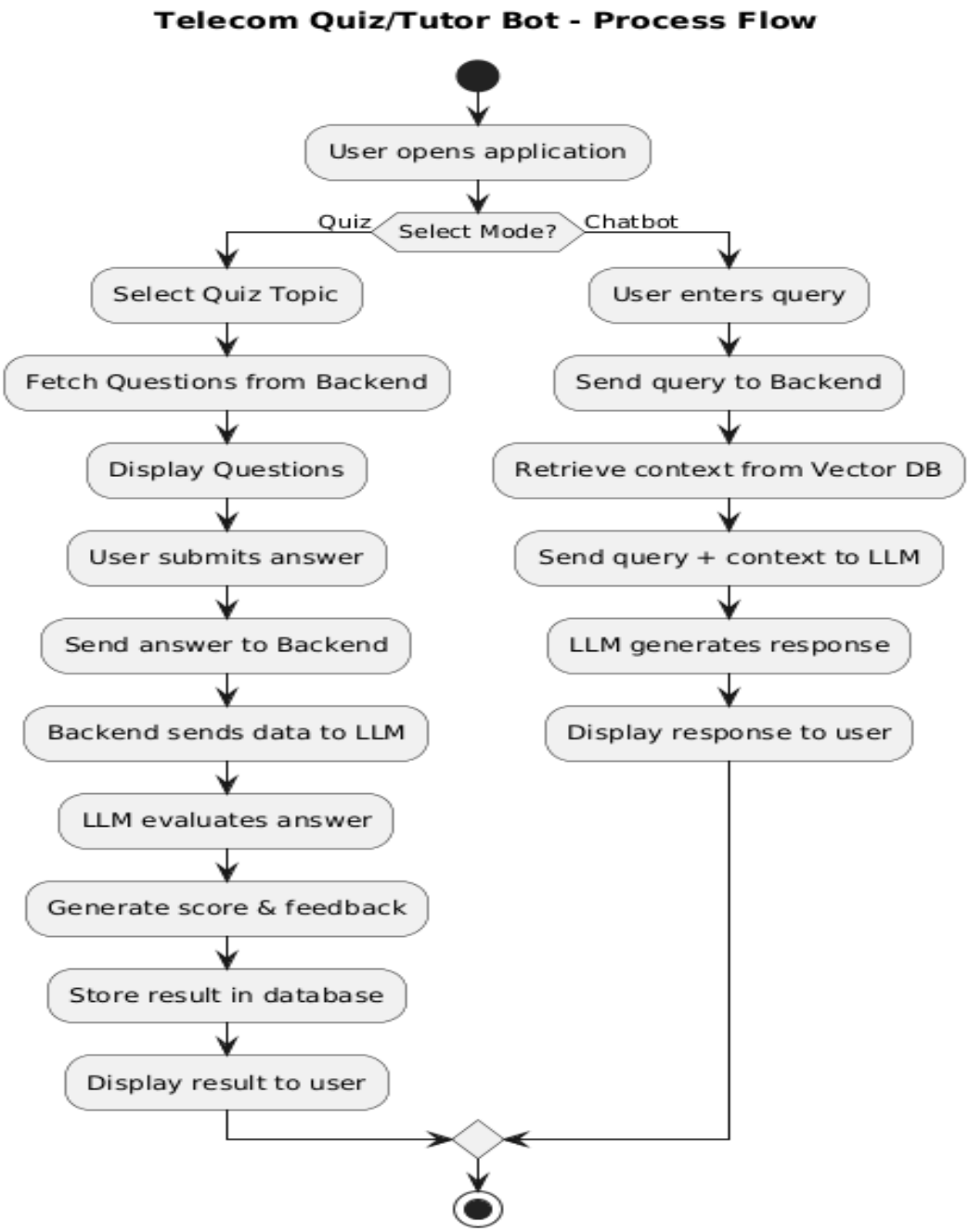
2.2 Process Flow

The process flow of the Telecom Quiz/Tutor Bot describes how the system handles user interactions for both quiz and chatbot functionalities.

When a user selects the quiz mode, the system fetches relevant questions from the backend and displays them on the interface. The user submits answers, which are sent to the backend for processing. The backend forwards the response to the Large Language Model (LLM), which evaluates the answer, generates a score, and provides feedback. The result is then stored and displayed to the user.

In chatbot mode, the user enters a query related to telecom topics. The query is sent to the backend, which retrieves relevant context from the vector database. The query along with the context is passed to the LLM, which generates an appropriate response. The response is then displayed to the user.

This flow ensures smooth interaction, real-time evaluation, and intelligent response generation within the system.



2.3 Information Flow

The information flow in the Telecom Quiz/Tutor Bot describes how data moves between different components of the system during user interaction.

When a user interacts with the system through the frontend, the request is sent to the backend API. For quiz functionality, user answers are forwarded

to the processing layer, where they are evaluated using a Large Language Model (LLM). The LLM returns a score and feedback, which are then stored in the database and displayed to the user.

For chatbot functionality, user queries are sent to the backend, which retrieves relevant information from the vector database. This context, along with the user query, is passed to the LLM to generate a meaningful response. The response is then returned to the frontend and displayed to the user.

Thus, the information flows seamlessly between the frontend, backend, AI processing layer, and data storage, ensuring efficient and real-time system performance.

2.4 Component Design

The Telecom Quiz/Tutor Bot system is composed of multiple components, each responsible for a specific functionality. The modular design ensures easy maintenance and scalability.

1. Frontend Component

- Provides the user interface for interaction.
- Includes quiz page, chatbot interface, and result dashboard.
- Sends user inputs (answers/queries) to the backend.

2. Backend Component (API Server)

- Built using FastAPI.
- Handles incoming requests and responses.
- Manages communication between frontend, AI processing, and database.

3. AI Processing Component (LLM Engine)

- Evaluates user answers using a Large Language Model (LLM).
- Generates scores, feedback, and explanations.
- Handles chatbot responses.

4. Vector Database Component

- Stores telecom-related knowledge and context.
- Used by chatbot to retrieve relevant information (RAG approach).

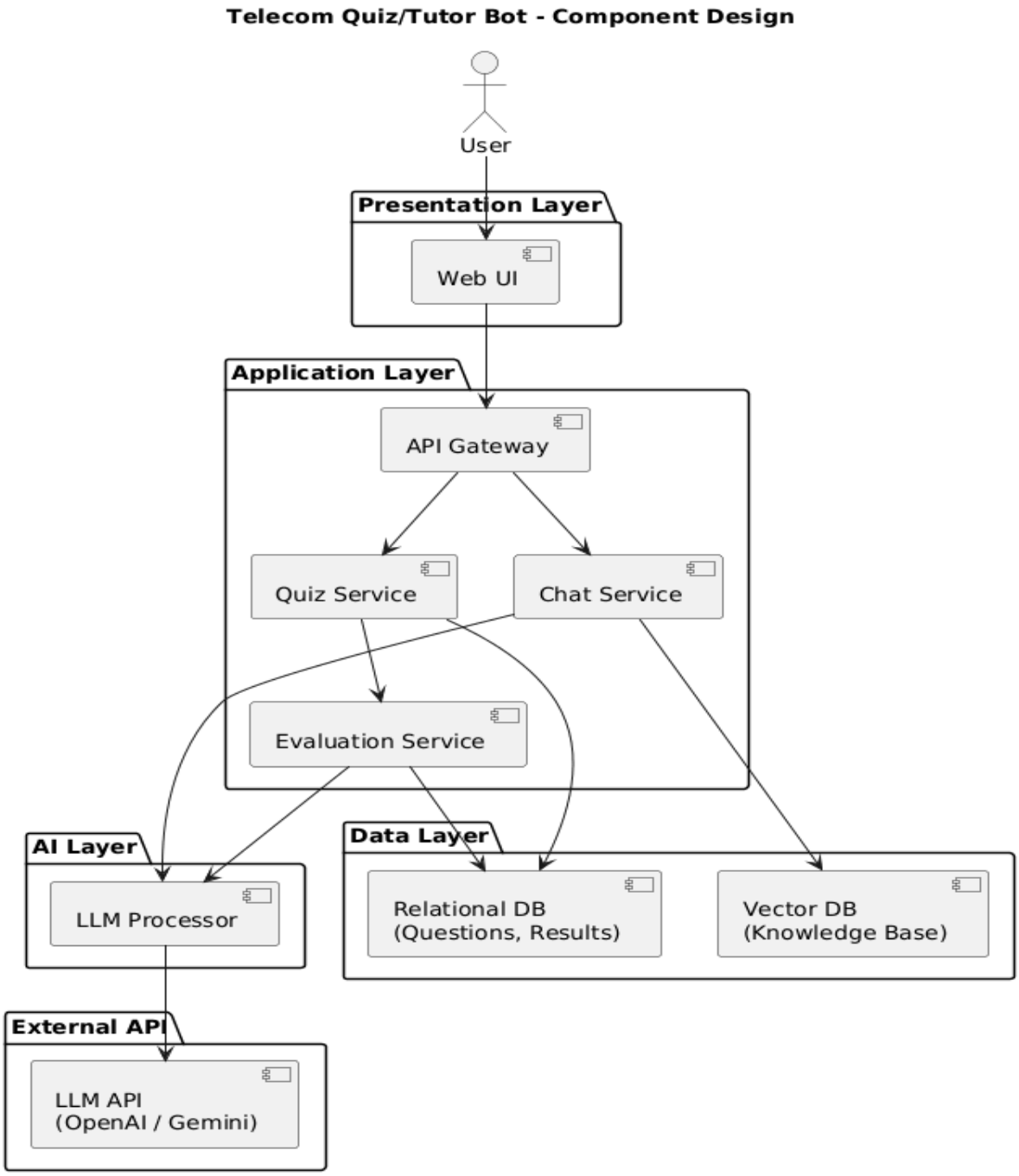
5. Data Storage Component

- Stores quiz questions, user responses, and performance data.

- Maintains user progress and results.

6. External LLM Service

- Provides AI capabilities through APIs (e.g., OpenAI/Gemini).
- Used for answer evaluation and chatbot response generation.



2.5 Key Design Consideration

The design of the Telecom Quiz/Tutor Bot system is based on several important factors to ensure efficiency, scalability, and reliability.

Key Considerations

1. **Scalability**
   The system is designed to handle multiple users and can be extended with additional quiz topics and features without affecting performance.

2. **AI-basedEvaluation**
   The use of a Large Language Model (LLM) enables intelligent and contextual evaluation of user answers instead of simple rule-based checking.

3. **Performance**
   FastAPI ensures quick processing of requests and real-time response generation for both quiz and chatbot functionalities.

4. **Accuracy**
   The integration of LLM and vector database improves the accuracy of feedback and chatbot responses.

5. **Maintainability**
   The modular architecture (frontend, backend, AI layer, data layer) makes the system easy to update and maintain.

6. **User Experience**
   Personalized feedback and interactive learning improve user engagement and understanding.

7. **Extensibility**
   New features like voice input, analytics dashboard, or additional domains can be easily added.

8. **Security**
   Input validation and controlled API access help prevent invalid or malicious inputs.

2.6 API Catalogue

The Telecom Quiz/Tutor Bot provides a set of REST APIs to handle quiz operations, answer evaluation, and chatbot interactions. These APIs are built using FastAPI and enable communication between the frontend and backend system.

1. GET /

Returns system status and welcome message.

Request Type: GET

Response Example:

```
{
  "message": "Welcome to Telecom Quiz Tutor Bot",
  "status": "active"
}
```

2. GET /questions

Fetches quiz questions based on selected topic.

Request Type: GET

Request Example:

```
{
  "topic": "data plans"
}
```

Response Example:

```
[
  {
    "id": 1,
    "question": "What is prepaid plan?",
    "topic": "plans"
  }
]
```

3. POST /submit_answer

Submits user answer for evaluation.

Request Type: POST

Request Body:

```
{
  "question_id": 1,
```

"user_answer": "Prepaid has no monthly bill"
}

Response Example:

{
"score": 1,
"feedback": "Correct, but missing validity explanation"
}

4. POST /chat

Handles chatbot queries.

Request Type: POST

Request Body:

{
"query": "What is 5G?"
}

Response Example:

{
"response": "5G is the fifth generation of mobile networks offering high speed and low latency."
}

5. GET /result

Returns user performance and score.

Request Type: GET

Response Example:

{
"score": 8,
"accuracy": "80%"
}

3. References

- OpenAI API Documentation – https://platform.openai.com/docs
- Google Gemini API Documentation – https://ai.google.dev/
- FastAPI Documentation – https://fastapi.tiangolo.com/
- Python Official Documentation – https://docs.python.org/3/